

VPA A2 S2 Weeks 1 – 9: Practical Programming Project

Victoria Park Academy — A2 Level Computer Science

Semester 2, Weeks 1 – 9: Practical Programming Project (Paper 4 Preparation)

Duration: 9 Weeks (Weeks 1 – 9) | Assessment: Practical Programming Project (NEA-style)

Language: Python 3.12 | Tools: VS Code, Git, GitHub, Python Standard Library

Objective: Students design, develop, test, and document a substantial programming project to demonstrate mastery of practical programming skills.

Project Overview

The Practical Programming Project is the capstone assessment of A-Level Computer Science. It requires students to:

Identify a real-world problem that can be solved computationally.

Analyse requirements and design an appropriate solution using algorithms and data structures.

Develop a working solution in Python, applying structured programming and OOP principles.

Rigorously test the solution using normal, boundary, and erroneous data.

Produce comprehensive technical documentation and a user guide.

Evaluate the solution against original objectives and suggest improvements.

☐ Academic Integrity: All code must be the student's own work. Plagiarism detection software will be used. Students must maintain a development log and version control history as evidence of independent work.

Week-by-Week Breakdown

Week 1: Project Launch & Problem Identification

Learning Objective: Students can identify a suitable problem and justify its computational solution.

Lesson Content

Introduction to the project requirements and assessment criteria.

Brainstorming session: potential project ideas (e.g., student management system, inventory tracker, quiz application, simulation, game with educational value).

Feasibility criteria: scope, complexity, data requirements, user interaction.

Project proposal template: problem statement, proposed solution, success criteria.

Differentiation

Bronze: Choose from a list of 5 pre-approved project briefs with clear requirements.

Silver: Adapt one of the pre-approved briefs with a personal twist.

Gold: Propose an original project idea; teacher approval required.

☐ Deliverable: Completed Project Proposal Form (problem statement, proposed solution, 5+ success criteria)

I have identified a real-world problem

I can explain why a computer program is the right solution

I have written at least 5 measurable success criteria

My teacher has approved my project idea

Week 2: Requirements Analysis & Research

Learning Objective: Students can gather, analyse, and document functional and non-functional requirements.

Lesson Content

Stakeholder identification: who will use the system?

Interview techniques and questionnaire design for gathering requirements.

Functional requirements (what the system must do) vs. non-functional requirements (how it must perform).

MoSCoW prioritisation: Must have, Should have, Could have, Won't have.

Research: existing solutions, libraries, APIs, and algorithms relevant to the project.

Practical Activity

Conduct a stakeholder interview (role-play with a peer or teacher). Record answers and extract requirements.

☐ Deliverable: Requirements Specification Document (functional + non-functional, MoSCoW prioritised, research summary)

I have identified all stakeholders

I have listed functional requirements (minimum 8)

I have listed non-functional requirements (minimum 4)

I have prioritised requirements using MoSCoW

I have researched existing solutions and noted what I will do differently

Week 3: System Design — Algorithms & Data Structures

Learning Objective: Students can design algorithms and select appropriate data structures for their solution.

Lesson Content

Top-down design: breaking the problem into modules/subroutines.

Algorithm design using pseudocode (Cambridge standard) and flowcharts.

Data structure selection: 1D/2D arrays, records, files, linked lists, stacks, queues, trees, hash tables.

Justifying data structure choices: time complexity, space complexity, ease of implementation.

Object-oriented design: class diagrams, inheritance, composition, encapsulation.

Practical Activity

Draw a class diagram for the project. Write pseudocode for the three most complex algorithms.

☐ Deliverable: Design Document — Module hierarchy diagram, class diagram, pseudocode for all major algorithms, data dictionary, data structure justification

I have created a module hierarchy diagram

I have drawn a class diagram (if using OOP)

I have written pseudocode for every major algorithm

I have created a data dictionary with variable names, types, and purposes

I can justify why I chose each data structure

Week 4: Development — Setting Up & Core Functionality

Learning Objective: Students can set up their development environment and implement core functionality.

Lesson Content

Setting up VS Code, Python 3.12, and Git version control.

Git workflow: init, add, commit, branch, merge. Maintaining a development log.

Code structure: main program loop, menu systems, input validation.

Implementing the "Must have" requirements first.

Defensive programming: input validation, error handling (try-except), range checks.

Practical Activity

Initialise a Git repository. Write the main program structure and implement the first two functional requirements.

☐ Deliverable: Working prototype with core functionality (Git repository link, development log, code comments)

I have initialised a Git repository and made at least 5 commits

I have a working main menu or program loop

I have implemented all "Must have" requirements

My code includes input validation for all user inputs

I have written a development log entry for each session

Week 5: Development — Advanced Features & Data Persistence

Learning Objective: Students can implement file handling, data persistence, and advanced features.

Lesson Content

File handling: text files, CSV files, JSON files. Reading, writing, appending.

Data persistence: saving user data between sessions.

Searching and sorting algorithms: linear search, binary search, bubble sort, insertion sort, quicksort.

Implementing "Should have" and "Could have" requirements.

Code refactoring: improving readability, efficiency, and structure.

Practical Activity

Implement file saving/loading. Add a search feature and a sort feature. Refactor code to reduce repetition.

☐ Deliverable: Enhanced program with data persistence, search/sort functionality, and at least 70% of requirements implemented

My program can save data to a file and load it correctly

I have implemented at least one search algorithm

I have implemented at least one sort algorithm

I have refactored my code to remove duplication

At least 70% of my requirements are now working

Week 6: Testing — Test Plans & Evidence

Learning Objective: Students can design and execute a comprehensive test plan using normal, boundary, and erroneous data.

Lesson Content

Test planning: what to test, how to test, expected vs. actual results.

Types of test data: normal (valid), boundary (extreme valid), erroneous (invalid).

Black-box testing: testing without looking at code.

White-box testing: testing with knowledge of code paths (statement coverage, branch coverage).

Integration testing: testing modules working together.

User acceptance testing: testing against success criteria with a stakeholder.

Practical Activity

Write a test plan with at least 15 tests (5 normal, 5 boundary, 5 erroneous). Execute all tests and record screenshots.

☐ Deliverable: Complete Test Plan (test number, description, test data, expected result, actual result, screenshot, pass/fail)

I have written at least 15 tests

I have tested with normal, boundary, and erroneous data

I have included screenshots as evidence

I have recorded expected vs. actual results for every test

Any failed tests have been fixed and re-tested

Week 7: Documentation — Technical Guide & User Guide

Learning Objective: Students can produce clear, comprehensive technical and user documentation.

Lesson Content

Technical documentation: system architecture, data structures, algorithms, code listings with comments, variable tables.

User guide: installation instructions, system requirements, step-by-step usage, screenshots, troubleshooting, FAQs.

Documentation standards: clarity, completeness, appropriate audience, professional presentation.

Annotating code: inline comments, docstrings, module descriptions.

Practical Activity

Draft the technical guide and user guide. Peer review with a partner: can they understand how to use your system?

☐ Deliverable: Technical Guide (system design, data dictionary, annotated code) + User Guide (installation, usage, screenshots, troubleshooting)

My technical guide explains the system architecture

I have included a full data dictionary

My code has meaningful comments and docstrings

My user guide has installation instructions

My user guide has step-by-step instructions with screenshots

A peer can use my system without asking me questions

Week 8: Evaluation & Maintenance Plan

Learning Objective: Students can critically evaluate their project and plan for future maintenance.

Lesson Content

Evaluation against success criteria: which criteria were met? Which were partially met? Which were not met? Why?

Evaluation of the solution: strengths, limitations, usability, performance, reliability.

Evaluation of the development process: what went well? What would you do differently?

Maintenance plan: corrective, adaptive, perfective maintenance. Future enhancements.

Client/stakeholder feedback: gathering and responding to user feedback.

Practical Activity

Conduct a stakeholder review. Gather written feedback. Write the evaluation section.

☐ Deliverable: Evaluation Report (success criteria review, strengths/limitations, process evaluation, stakeholder feedback, maintenance plan, future enhancements)

I have evaluated every success criterion

I have identified at least 3 strengths of my solution

I have identified at least 3 limitations and explained why they exist

I have gathered stakeholder feedback and responded to it

I have written a maintenance plan with examples of corrective, adaptive, and perfective maintenance

I have suggested at least 3 future enhancements

Week 9: Final Polish, Project Review & Submission

Learning Objective: Students can finalise their project and present it for assessment.

Lesson Content

Final code review: readability, efficiency, comments, PEP 8 compliance.

Final documentation review: completeness, consistency, professional presentation.

Creating a project portfolio: organising all files, code, documentation, and evidence.

Submission checklist: ensuring all required components are present.

Peer presentation: 5-minute demonstration of the project to the class.

Practical Activity

Complete the submission checklist. Present the project to peers. Submit the final portfolio.

☐ Deliverable: Complete Project Portfolio (all code, all documentation, all evidence, Git repository, presentation)

I have checked my code against PEP 8 style guidelines

I have proofread all documentation for spelling and grammar

I have included all required sections: analysis, design, development, testing, documentation, evaluation

I have a complete Git commit history

I have presented my project to the class

I have submitted my portfolio before the deadline

Project Assessment Criteria

Section	Max Marks	What the Examiner Looks For
Analysis	12	Clear problem statement, well-researched, comprehensive requirements, appropriate success criteria
Design	15	Logical decomposition, appropriate algorithms, justified data structures, clear pseudocode/flowcharts
Development	20	Working solution, efficient code, good structure, appropriate use of programming constructs, evidence of iteration
Testing	12	Comprehensive test plan, normal/boundary/erroneous data, evidence (screenshots), error resolution
Documentation	8	Clear technical guide, usable user guide, well-commented code
Evaluation	8	Critical review against success criteria, honest limitations, stakeholder feedback, maintenance plan
Total	75	

Mock Exams During Project Phase

Mock 2 (Week 3): Paper 1 Theory — 1h 45m, 75 marks. Baseline to identify theory gaps while project work continues.

Mock 3 (Week 6): Paper 2 Programming — 2h, 75 marks. Tests programming under timed conditions alongside project development.

Resources

Python 3.12 Documentation (docs.python.org)

Git Tutorial (git-scm.com/doc)

VS Code Python Extension Guide

Cambridge A-Level Computer Science (9618) Syllabus — Paper 4 requirements

PEP 8 Style Guide (pep8.org)

VPA Bilingual Glossary (150 Terms) — Supporting Documents

Document Control: Version 1.0 | Created: January 2026 | Owner: VPA Computer Science Department

Next Review: June 2026 (post-exam analysis)